MyCodeSchool

Code review — slot-mate

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-19 — sha `f11db8a`

Scop: feature-ul 1 din plan, Categorii (`fe42b0b` „Category 1.0"), testul de controller (`f11db8a` „CategoryControllerTest errors") și configurarea.

Metodă: build cu Gradle, apoi API-ul tău pornit pe un MySQL 8.4 real (container local, `ddl-auto: update` din `application.yaml` -ul tău) și fiecare endpoint apelat cu `curl`. Codul tău nu a fost atins; unde un bug ascundea altul, l-am ocolit pe o copie separată.

Poarta de compilare: ✅ `main` se compilează · 🔴 testele nu (`compileTestJava` pică, B5).

Înainte de orice — de ce nu pornește din IntelliJ: `Could not find or load main class ro.mycode.slotmateapi.SlotMateApiApplication` . Codul e bun; configurația de rulare din IntelliJ a rămas pe pachetul din Spring Initializr, iar tu ai mutat clasa în `ro.mycode.app` . 💡 Run → Edit Configurations: ce scrie la *Main class*? Sau șterge configurația și pornește din săgeata verde de lângă `main` .

Ce am primit la rulare:

Cerere	Răspuns	Ce trebuia (contractul tău `openapi.yaml`)
`GET /all` pe tabel gol	500	200 `[]`
`POST /add {"name":"Frizerie"}`	500	201
`POST /add` același nume a doua oară	500	409
`POST /add {"name":""}`	500	400
`PATCH /update/Nimic`	500	404 (în afara contractului)
`DELETE /delete/Nimic`	500	404 (în afara contractului)
`GET /all` cu 2 categorii (introduse din SQL)	✅ 200	✅

Cerere	Răspuns	Ce trebuia (contractul tău `openapi.yaml`)
`PATCH /update/Frizerie` → `Barber`	✅ 200	✅
`DELETE /delete/Dentist`	✅ 200	✅

Partea care merge e logica de citire, modificare și ștergere. Toate 500-urile vin din 5 cauze.

🔴 Critice

B1 — `Category.java:19–20` : nicio categorie nu se poate crea

Dovada: orice `POST /add` → 500, în log `Identifier of entity 'Category' must be manually assigned before calling 'persist()'`.

Mecanism: `@Id` doar marchează cheia primară. Cine pune valoarea în ea? Fără o strategie de generare, Hibernate presupune că o pui tu, în Java, înainte de `save`. Mapper-ul tău nu pune `id`, deci Hibernate refuză insert-ul înainte să ajungă la MySQL.

💡 Uită-te la `User`, `Provider` sau `Slot`: acolo id-ul se generează. Ce adnotare are în plus? 💡 Atenție: `ddl-auto: update` adaugă coloane, dar nu modifică una existentă. Tabela `categories` creată deja la tine are `id` fără `AUTO_INCREMENT`; după fix, șterge-o o dată.

B2 — `CategoryController.java:35` : validarea din `CategoryCreateRequest` nu rulează

Dovada (pe copia cu B1 ocolit): `POST {"name":""}` și `POST {}` → 500, în log `ConstraintViolationException ... during persist time`, cu mesajul din **entitate**, nu din DTO.

Mecanism: `@NotBlank` pe un record nu face nimic singur; cineva trebuie să ceară validarea. Pe un `@RequestBody`, Spring o face doar dacă îi spui, înainte de a intra în metodă. Fără asta, cererea trece de controller, iar valoarea goală o prinde abia Hibernate, la `persist`, pe `@NotBlank` din `Category`. Atunci e prea târziu pentru un 400: eroarea a apărut deja în mijlocul serviciului.

💡 Ce adnotare pui lângă `@RequestBody` ca Spring să valideze înainte? 💡 Același lucru lipsește la `updateCategory` (`:47`).

B3 — `CategoryCommandServiceImpl.java:32–33` : `update` pe un nume inexistent → 500

Dovada: `PATCH /update/Nimic` → 500 `NoSuchElementException: No value present`.

Mecanism: `getCategoryByName` întoarce un `Optional`, iar un `Optional` nu e niciodată `null`: când nu găsește nimic, întoarce o cutie goală. Deci `== null` e mereu fals, garda nu se declanșează niciodată, iar `.get()` pe cutia goală aruncă excepția.

💡 Ce metodă din `Optional` aruncă direct excepția ta când cutia e goală? Așa scrii un singur apel în loc de două. 💡 + același tipar dublu (verifici, apoi cauți încă o dată) la `deleteCategory:43–44` și `getAllCategories:19–20`.

B4 — `CategoryNotFoundException` iese ca 500, nu 404

Dovada: `DELETE /delete/Nimic` → 500; `GET /all` pe tabel gol → 500.

Mecanism: o excepție aruncată din serviciu urcă până la Spring. Dacă nimeni nu i-a spus ce cod HTTP îi corespunde, Spring trimite 500, „eroare a serverului". Iar `GET /all` pe o listă goală nu e o eroare deloc: „nu există categorii încă" e un răspuns valid.

💡 Unde îi spui lui Spring „excepția X → 404"? Caută `@RestControllerAdvice` (în plan: ProblemDetail). 💡 La `getAllCategories`: ce ar trebui să primească clientul când nu sunt categorii? 💡 Tot aici intră și dublura: `POST` cu un nume existent → 500 `Duplicate entry 'Frizerie'` (la fel la `PATCH` spre un nume existent). În contract e 409.

B5 — `CategoryControllerTest.java`: testul nu se compilează și nu pornește

Am reparat pe copie câte o eroare și am rulat din nou; în ordinea în care le întâlnești:

1. `:41` `status.isOk()` → `cannot find symbol`. `status()` e o metodă statică. 💡 De unde vine `get(...)`, pe care îl folosești fără prefix? Fă la fel pentru `status()`.
2. `:4` `jsonPath` e importat din `MockRestRequestMatchers`: clasa e pentru testele în care aplicația ta cheamă alt server, nu se potrivește cu `andExpect` din MockMvc. 💡 Caută `jsonPath` în aceeași clasă cu `status()`.
3. `:1` pachetul `ro.mycode.slotmateapi...` → `initializationError`. `@WebMvcTest` caută clasa cu `@SpringBootApplication` urcând din pachetul testului; din `ro.mycode.slotmateapi` nu ajunge niciodată la `ro.mycode.app`. 💡 Uită-te unde stă acum `SlotMateApiApplicationTests`.
4. `:6`, `:25–26` `ObjectMapper` din `com.fasterxml` → `NoSuchBeanDefinitionException`. Spring Boot 4 folosește Jackson 3 (`tools.jackson.databind`). La un GET nici nu-ți trebuie.
5. `:43` `$.category.ResponseList[1]` → `No value at JSON path`. 💡 Compară cu rândul de deasupra, literă cu literă.

După toate cinci: ✅ testul trece. Pe drum, `@WebMvcTest` și `@MockitoBean` pe ambele servicii sunt alese bine.

🟡 Importante

- **M1 — rutele nu respectă contractul tău** (`openapi.yaml`): ai `/api/v1/categories/all`, `/add`, `/delete/{name}`, `/update/{email}`; contractul are `GET` + `POST /api/categories`. În REST verbul e metoda HTTP, nu e în URL. `GET` întoarce în contract un array, nu `{categoryResponseList: [...]}`.
- **M2 — `CategoryController.java:40-50`:** `update` și `delete` nu sunt în contract (F1 are doar „listează" și „adaugă"). Le poți păstra, dar trec în `openapi.yaml` și `FEATURES.pdf`. Variabila de cale de la `update` se numește `email`, deși e numele categoriei.
- **M3 — `@Controller` în loc de `@RestController`** (`:15`). Merge doar pentru că întorci `ResponseEntity`. `/v3/api-docs` iese cu `paths: {}` (n-am confirmat că asta e cauza).
- **M4 — config dublat:** `application.properties` (pus de mine în `a63b837`) și `application.yaml` (al tău) setează amândouă `spring.datasource.*`. La aceeași cheie câștigă `.properties`, deci valorile tale implicite din yaml sunt ignorate. 💡 Păstrează un singur loc: șterge cele 3 linii `datasource` din `.properties`. Tot acolo: `DB_PASWORD` (`application.yaml:5`) are un S lipsă.
- **M5 — `application-test.yaml` (H2) nu e folosit:** nu există profilul `test` și nici dependența H2; testele merg pe MySQL prin Testcontainers.
- **M6 — entitățile din `models` / `Category 1.0`** (citite, nerulate: nu există încă endpoint-uri): `@NotBlank` pe relații în `Provider` / `Offering` (pe obiecte e `@NotNull`); `cascade = ALL` pe `@ManyToOne category` din `Provider` (ștergerea unui furnizor ar șterge categoria tuturor); `Offering.price` e `Double` (bani → `BigDecimal`); coloana `adress`; `User.createdAt` fără `@Builder.Default`.

🟢 Cleanups

- **C1:** `CategoryMapper.java:20`: variabila `name` nu e folosită.
- **C2:** `CategoryRepository` are două metode pentru același lucru (`findByName` → `List`, `getCategoryByName` → `Optional`); numele e `unique`, deci ajunge cea cu `Optional`.
- **C3:** mesajele de validare sunt în română; regula proiectului: în engleză, ca în `ExceptionConstants`. Comentariile din `Booking` / `Slot` se scot.
- **C4:** `@Transactional` din `jakarta.transaction`; în Spring folosește-l pe cel din `org.springframework.transaction.annotation`.

Întrebări

1. După ce repari B1, de ce `POST` tot nu merge la tine local până nu ștergi tabela `categories`?
2. Un `POST {"name":""}`: ce răspuns trebuie să vadă clientul și în ce punct al drumului (controller → serviciu → Hibernate) trebuie oprită cererea?
3. De ce `getCategoryByName(name) == null` nu poate fi niciodată adevărat?